

A verdade nasce onde o pensamento é livre.

A Crise do COBOL Não É uma Crise do COBOL

Publicado em 2026-10-03 11:38:00



FC-CHRONIC-NEWS · TECNOLOGIA ·
MAINFRAME · ARQUITECTURA DE SISTEMAS ·
INTELIGÊNCIA ARTIFICIAL

Blogue Fragmentos do Caos



A verdade nasce onde o pensamento é livre.

O verdadeiro legado dos mainframes não está na sintaxe: está na arquitectura, nas regras de negócio, nas dependências e na memória institucional executável.

Nota de Abertura

O mundo tecnológico fala de uma “crise do COBOL”. Talvez esteja a olhar para a camada errada. A sintaxe pode ser ensinada; décadas de arquitectura, dependências, regras de negócio e memória operacional não cabem num manual de linguagem.

Tese técnica

COBOL « **conhecimento da aplicação** «
conhecimento do sistema «
conhecimento do negócio «
conhecimento arquitectural. Modernizar
não é traduzir source code: é recuperar o
modelo do sistema, preservar invariantes e

Blogue Fragmentos do Caos



A verdade nasce onde o pensamento é livre.

A linguagem é apenas a superfície

O debate sobre os sistemas legados continua demasiadas vezes preso a uma palavra: COBOL.

É compreensível. É a parte visível. Há milhões de linhas de código com décadas de existência, equipas envelhecidas e sistemas críticos que continuam a processar operações bancárias, seguros, administração pública, transportes e outras funções essenciais.

Mas confundir o problema com a linguagem é confundir o mapa com o território.

Um programador competente pode aprender a sintaxe de COBOL. O que não se aprende em algumas semanas é a história operacional de uma aplicação que foi alterada durante trinta, quarenta ou cinquenta anos.

O conhecimento decisivo não é saber escrever um PERFORM ou interpretar uma PIC clause.

Blogue Fragmentos do Caos



A verdade nasce onde o pensamento é livre.

VSAM constitui a fonte de verdade; em que sequência o JCL executa o batch nocturno; e o que acontece ao resto do sistema quando um job termina de forma anómala.

Isso já não é apenas programação.

É engenharia de sistemas.

O verdadeiro legado

Um grande sistema mainframe não é simplesmente uma colecção de programas COBOL.

É um ecossistema.

COBOL, PL/I ou Assembler podem coexistir com JCL, CICS, Db2, VSAM, filas, ficheiros, interfaces, rotinas operacionais, calendários de batch, regras de recuperação, procedimentos humanos e integrações construídas ao longo de décadas.

Por isso, a hierarquia do conhecimento relevante é aproximadamente esta:

Blogue Fragmentos do Caos



A verdade nasce onde o pensamento é livre.

A linguagem é necessária para ler o código.

A arquitectura é necessária para compreender o que acontece quando se muda o código.

E essa diferença é gigantesca.

Memória institucional executável

Os sistemas legados possuem uma característica extraordinária: acumulam decisões organizacionais.

Uma regra fiscal alterada em 1997 pode continuar representada numa condição. Uma excepção comercial negociada há vinte anos pode sobreviver numa tabela. Uma limitação de um sistema anterior pode ter originado uma sequência de processamento que ninguém hoje sabe justificar.

O analista que conhecia a origem da regra reformou-se.

O programador que escreveu o módulo saiu da empresa.

Blogue Fragmentos do Caos



A verdade nasce onde o pensamento é livre.

Todos os dias.

Neste sentido, um grande sistema legado tornou-se uma espécie de **memória institucional executável**.

Eliminar código sem recuperar primeiro essa memória é parecido com restaurar um edifício histórico destruindo as paredes antes de descobrir quais suportam o telhado.

As perguntas que importam

Perante um sistema destes, o engenheiro experiente não começa necessariamente por perguntar: “Como converto isto para Java?”

Começa por perguntar:

Onde estão os estados persistentes?

Quais são as dependências explícitas e implícitas?

Onde estão as fronteiras transaccionais?

Qual é a fonte de verdade de cada entidade?

Blogue Fragmentos do Caos



A verdade nasce onde o pensamento é livre.

Como funciona rollback e recuperação?

Que componentes são partilhados por aplicações aparentemente independentes?

Onde estão os pontos únicos de falha?

Que interfaces possuem contratos reais e quais dependem apenas de comportamento histórico?

Estas perguntas pertencem à arquitectura de sistemas.

E continuam válidas independentemente de o código ser COBOL, Java, C#, Python ou qualquer linguagem que venha a substituí-las.

O erro COBOL → Java

É aqui que a actual vaga de inteligência artificial pode produzir simultaneamente uma extraordinária oportunidade e uma perigosa ilusão.

A ilusão seria formular o problema assim:

Blogue Fragmentos do Caos



A verdade nasce onde o pensamento é livre.

Uma tradução sintacticamente correcta pode preservar comportamento local e, ainda assim, deixar intacta uma arquitectura inadequada — ou, pior, ocultar dependências que antes eram pelo menos conhecidas operacionalmente.

O problema correcto é muito mais amplo:

Legacy system → descoberta → modelo funcional → modelo de dados → dependências → regras de negócio → invariantes → arquitectura-alvo → migração incremental → validação.

Só depois surge a questão da linguagem de implementação.

E talvez essa seja uma das decisões menos difíceis de todo o processo.

A indústria já começou a perceber isto

É significativo que as próprias plataformas modernas de transformação estejam a evoluir nesta direcção.

Blogue Fragmentos do Caos



A verdade nasce onde o pensamento é livre.

blueprint do landscape aplicacional. A sua arquitectura de modernização utiliza análise prévia para visualizar relações entre programas e identificar serviços de negócio antes de qualquer transformação selectiva para Java.

A AWS segue uma lógica semelhante no AWS Transform para mainframe: análise da codebase, documentação técnica, extracção de lógica de negócio, decomposição por domínios, estudo de dependências, planeamento da sequência de migração e só então refactoring.

Isto é importante.

As ferramentas que inicialmente poderiam ser apresentadas como tradutores de COBOL estão progressivamente a tornar-se ferramentas de **compreensão de sistemas**.

Porque o mercado descobriu aquilo que muitos engenheiros de sistemas já sabiam:

converter código é relativamente fácil; compreender as consequências da conversão é o problema difícil.

Blogue Fragmentos do Caos



A verdade nasce onde o pensamento é livre.

Não apenas como gerador de Java.

Mas como arqueólogo de software.

Imagine-se um modelo especializado capaz de ingerir COBOL, JCL, copybooks, esquemas Db2, estruturas VSAM, mapas CICS/BMS, documentação, logs, histórico de alterações, incidentes e contratos de interfaces.

Em vez de produzir imediatamente código novo, construiria progressivamente um grafo semântico do sistema:

negócio → regras → transacções → batch → programas → dados → interfaces → sistemas externos.

Cada nó possuiria relações.

Cada relação teria evidência.

Cada regra poderia ser rastreada até ao código que a implementa e aos processos que dela dependem.

Blogue Fragmentos do Caos



A verdade nasce onde o pensamento é livre.

Por exemplo:

“Se remover este módulo, que processos podem deixar de funcionar?”

“Que programas alteram este campo?”

“Que regra de negócio explica este cálculo?”

“Quais são todos os consumidores desta tabela?”

“Que jobs precisam de terminar antes desta transacção ficar disponível?”

“Mostra todas as dependências que atravessam este domínio de negócio.”

Isto é muito mais valioso do que tradução automática.

É recuperação de conhecimento.

Do source code ao knowledge graph

A arquitectura de uma plataforma deste tipo poderia organizar-se em várias camadas.

Blogue Fragmentos do Caos



A verdade nasce onde o pensamento é livre.

Acima surgiria a análise estática e dinâmica: call graphs, acessos a dados, fluxos transaccionais, sequências batch e dependências temporais.

Depois viria uma camada semântica: regras de negócio, entidades, domínios, invariantes e contratos.

Finalmente, um knowledge graph permitiria navegar do conceito empresarial até à implementação física — e no sentido inverso.

A IA deixaria então de responder apenas sobre código.

Passaria a responder sobre **o sistema**.

Essa diferença pode determinar o sucesso ou o fracasso de uma modernização.

O humano continua no centro

Há, contudo, um limite fundamental.

Nenhum LLM pode garantir que recuperou uma intenção que nunca foi documentada e que não pode ser inferida inequivocamente dos artefactos disponíveis.

Blogue Fragmentos do Caos



A verdade nasce onde o pensamento é livre.

Uma condição pode revelar que determinado cliente recebe tratamento diferente. Pode não explicar por que razão essa exceção foi criada, se ainda é juridicamente necessária ou se corresponde a uma regra que deveria ter desaparecido há vinte anos.

Por isso, a modernização séria continuará a necessitar de engenheiros, arquitectos, operadores e especialistas do negócio.

A IA pode reduzir brutalmente o custo da arqueologia.

Não elimina a necessidade de julgamento.

Na realidade, pode torná-lo ainda mais importante.

A verdadeira crise de competências

Talvez devamos então reformular aquilo a que chamamos “escassez de programadores COBOL”.

Existe certamente um problema geracional de competências específicas.

Blogue Fragmentos do Caos



A verdade nasce onde o pensamento é livre.

Pessoas capazes de relacionar aplicação, sistema operativo, transacção, base de dados, comunicação, batch, disponibilidade, recuperação, desempenho e negócio.

Esse conhecimento não pertence a uma linguagem.

Pertence a uma forma de pensar.

E ironicamente, quanto mais avançamos para cloud, microservices, Kubernetes e inteligência artificial, mais precisamos dessa capacidade.

Porque sistemas distribuídos modernos não são menos complexos.

São complexos de maneira diferente.

**As linguagens envelhecem. A
arquitectura permanece.**

COBOL desaparecerá um dia.

Talvez muito mais tarde do que muitos imaginam.

Blogue Fragmentos do Caos



A verdade nasce onde o pensamento é livre.

Frameworks desaparecerão.

Clouds mudarão.

Modelos de IA que hoje parecem revolucionários serão peças de museu tecnológico.

Mas algumas perguntas sobreviverão:

Onde está o estado?

Quem depende de quem?

O que acontece quando isto falha?

Como recuperamos?

Quem é responsável pela verdade?

Que invariantes não podemos quebrar?

Qual é o impacto de mudar este componente?

Estas perguntas atravessam gerações tecnológicas.

Por isso talvez a chamada “crise do COBOL” esteja mal baptizada.

Blogue Fragmentos do Caos



A verdade nasce onde o pensamento é livre.

É uma crise de perda de memória arquitectural, conhecimento institucional e compreensão sistémica.

E é precisamente aí que a inteligência artificial pode prestar o seu maior serviço: não escrever mais depressa aquilo que já existe, mas ajudar-nos finalmente a compreender aquilo que construímos.

Porque modernizar um sistema não significa apenas mudar a linguagem em que ele fala.

Significa compreender suficientemente bem aquilo que ele sabe para podermos ensinar esse conhecimento ao sistema que virá depois.

Nota Editorial

Esta é uma crónica técnica e interpretativa. A tese central — de que o risco principal dos sistemas legados reside menos na sintaxe COBOL do que na perda de conhecimento arquitectural, regras de negócio e memória institucional — é uma interpretação dos autores

Blogue Fragmentos do Caos



A verdade nasce onde o pensamento é livre.

A documentação actual da IBM confirma uma abordagem que começa pela compreensão do landscape aplicacional, relações e blueprint antes da transformação selectiva. A AWS descreve igualmente processos de análise, documentação, extracção de lógica de negócio, decomposição por domínios, estudo de dependências, planeamento e refactoring. Estas fontes sustentam a distinção entre simples tradução de linguagem e modernização sistémica.

A proposta de utilizar IA para construir um grafo semântico que relacione negócio, regras, transacções, batch, código, dados e interfaces é uma arquitectura conceptual apresentada nesta crónica. Não é descrita como implementação específica de qualquer produto citado.

Referências técnicas internacionais

- IBM — Code Generation for Z: arquitectura de análise, refactoring e transformação selectiva

Blogue Fragmentos do Caos



A verdade nasce onde o pensamento é livre.

the Rosetta Stone for COBOL (2025)

- IBM — Business Rule Discovery no watsonx Code Assistant for Z
- AWS — AWS Transform for modernizing mainframe applications
- AWS — Modernization of mainframe applications: COBOL, JCL, CICS, Db2 e VSAM
- AWS — Mainframe modernization: decoupling patterns and dependencies
- AWS — AI-powered mainframe modernization: business domains, dependencies e wave planning

Francisco Gonçalves

com a colaboração, co-autoria editorial e investigação de **Augustus Veritas — AI Assistant**

Fragmentos do Caos · FC-Chronic-News · Outubro de 2026

Blogue Fragmentos do Caos



A verdade nasce onde o pensamento é livre.

No **Fragmentos do Caos**, a nossa equipa reúne experiência profissional de várias décadas em **arquitectura de sistemas, infra-estruturas informáticas e aplicações empresariais**, incluindo ambientes mainframe e ecossistemas aplicativos baseados em **COBOL**.

Conhecemos, por experiência prática, uma realidade que este artigo procura salientar: modernizar um sistema legado não consiste simplesmente em converter código para uma linguagem mais recente. É necessário compreender a **arquitectura, regras de negócio, fluxos transaccionais, processamento batch, estruturas de dados, interfaces, dependências e requisitos operacionais** que foram sendo acumulados ao longo de décadas.

É também nesta área que consideramos particularmente promissora a utilização de **Inteligência Artificial aplicada à análise e modernização de sistemas legados**: não apenas para geração ou conversão de código, mas para recuperação de conhecimento, documentação, identificação de dependências e reconstrução do mapa funcional e arquitectural das aplicações.

Blogue Fragmentos do Caos



A verdade nasce onde o pensamento é livre.

nos para uma primeira análise do problema.

Contacto: infos@softelabs.pt

 [GitHub Pages](#)

 [CodeBerg Pages](#)



Fragmentos do Caos: [Blogue](#) • [Ebooks](#) • [Carrossel](#)

 Esta página foi visitada ... vezes.

[Contactos](#)